Understanding SSH and SFTP

CS 407 Intro To Linux

November 12, 2025

1 Introduction

In this lab you will strengthen your understanding of ssh and sftp.

1. SSH - Secure SHell. Used for connecting to remote computers.
2. SFTP - Secure File Transfer Protocol. Used for sending and retrieving files to and from remote computers.

SSH allows you to do alot of configuration. We'll look at just a couple of things you can configure, and well use sftp to send some files back and forth to a droplet.

Create a brand new Debian 13 droplet now with ssh authentication (not password!) and then we'll begin.

2 Useful background information

By this point in the class you should be familiar with permissions and ownership of files and directories.

If you run ls -l you will see the permissions and ownership/group of files/directories. (TODO add picture)

2.1 ownership

All files / directories have an owner and a group.

In the ls -l output you see the owner and group for each file (add picture)

To change the ownership/group you can do

chown user:group file_or_directory

If you want to change the user/group RECURSIVELY for a directory and all it's contents, you do

chown -R user:group directory.

2.2 permissions

To change permissions you do chmod xyz file_or_dir

where xyz is a 3 digit number like 742, 600, 777, 000, etc.

The digits mean something special.

The first digit is for the user The second digit is for the group The last digit is for everyone else on the computer

Each digit is a sum of 4, 2, 1.

4 - read 2 - write 1 - execute

Show how read/write/execute work for files

For directories its a bit different 4 - 2 - 1-

For more info on permissions, see this link `https://github.com/melvyniandrag/LinuxClassRepo/blob/master/usersAndPermissions.md`

3 SSH Configuration

3.1 How SSH Works

There's no time for the full story, but here's a little information. On your client machine (e.g. your laptop, pc, macbook, etc) you have a folder **$HOME/.ssh** and in there you have an ssh keypair. In previous lectures we have used **ssh-keygen** to generate the key pair. One has file extension**.pub** and the other does not.

- id_rsa (or id_ed22519, etc.) is your private key
- id_rsa.pub (or another name, it is your private key with a .pub added) is your public key

TODO add image here of output of ls ∼/.ssh/

CANT ADD PIC NOW, MY PC HAS A CUSTOM SSH CONFIG THAT WILL CONFUSE STUDENTS

On your ssh server (your digital ocean droplet, for example), to be able to ssh in as a user (such as root), your public key must be on a line in the file

∼/.ssh/authorized_keys

The authorized_keys file can contain one public key for line for each public key that allows logging in.

So if the pub key on my laptop is

"id_rsa asdlgkjaheliuyth melvyn@pc"

and I want to ssh into my droplet like ssh root@192.334.223.445

then there must be a line in the file /root/.ssh/authorized_keys that contains my key.

If I want to ssh in to melvyn@374.234.323.456, then on machine 374.234.323.456, my public key must be a line in the file /home/melvyn/.ssh/authorized_keys

The permissions on the .ssh folder should be 700, the permissions on the authorized keys file must be 700, and the owner/group of both the directory and the file should be melvyn/melvyn.

3.2 daemons

A *daemon* (pronounced like demon) is a process that runs in the background doing work without user interaction, and there are many daemon processes that do important work on Linux computers, such as

- sshd - ssh daemon
- crond - runs cron jobs
- udevd - handles hardware events like when devices are plugged in or unplugged
- etc.

The ssh daemon **sshd** can be configured to control how ssh clients connect to your droplet, which is the ssh server.

3.3 /etc/ssh/sshd_config

The configuration file looks up a number of properties. The first match it finds is what it uses. So if you say "X" is true, but then later say it's false - sshd will think it's true. It uses the first match.

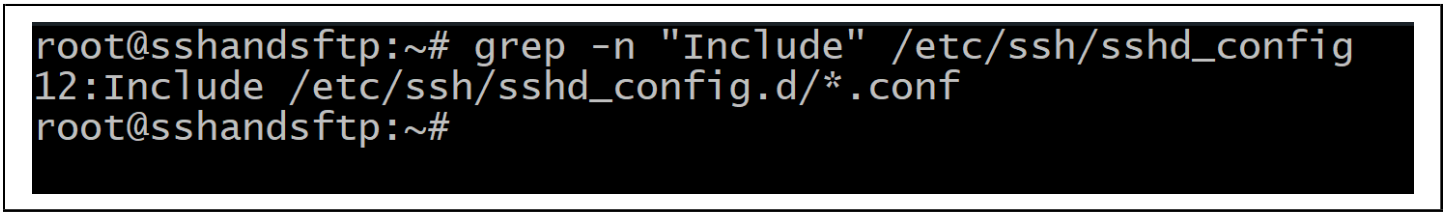

Figure 1: `Include`

Let's use **vim** to inspect the ssh daemon configuration file, **/etc/ssh/sshd_config**. Here I also show a few screenshots where I use `grep -n` to show you a linenumber with something important. Figure 1 shows that we include additional config files in the **/etc/ssh/sshd_config.d/** directory. Figure 2 shows that password authentication is off. But keep in mind, since we include the config files on line 12, but turn off password authentication on line 57, if an additional config file turns ON password authentication, this will stick. sshd uses the first match it finds.

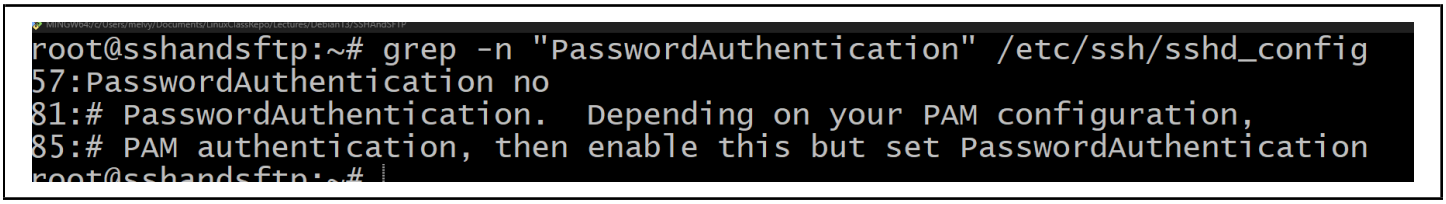

Figure 2: `Include`

3.4 Additional Config Files

See in figure 3 that there is an additional configuration file. This is the file that is included by the main configuration file.

Note that this file has a line turning off password authentication.

Also note that this file starts with a 50. The files are included in sorted order, so if you want to include some setting first, save it in a file called 40-something.conf or 25-somethingelse.conf.

Take some time now to inspect all of the files in **/etc/ssh/sshd_config.d**. Note that it is customary to distinguish the config file **sshd_config** from the directory **sshd_config.d** by adding the .d to the end of the directory.

Figure 3: `50-cloud-init.conf`

3.5 Activity 1

Create a new user on your machine

```
root@droplet$ adduser student
# set a password.
# otherwise, just accept defaults.
```

Attempt to ssh in to this use from your windows or mac computer. It should fail.

```
me@myPC$ ssh student@droplet
# error something something (Public Key Denied)
```

3.6 Activity 2

Turn on password authentication in your **/etc/ssh/sshd_config**. Then restart sshd. Then attempt to ssh in again as the student. It should fail, because the setting from **/etc/ssh/sshd_config.d/50-cloud-init.conf** is overriding the setting.

```
root@droplet$ # use vim to edit /etc/ssh/sshd_config
root@droplet$ grep "PasswordAuthentication" /etc/ssh/sshd_config
PasswordAuthentication yes
root@droplet$ systemctl restart sshd
root@droplet$ exit
me@myPC$ ssh student@droplet
# error something something (Public Key Denied)
```

3.7 Activity 3

This time, turn on PasswordAuthentication in the 50-cloud-init file. Then restart sshd and try to connect. It should now work!!

```
root@droplet$ # use vim to edit /etc/ssh/sshd_config.d/50-cloud-init.conf
root@droplet$ grep "PasswordAuthentication" /etc/ssh/sshd_config.d/50-cloud-init.conf
PasswordAuthentication yes
root@droplet$ systemctl restart sshd
root@droplet$ exit
me@myPC$ ssh student@droplet
# asks for your password, and you can get in!
```

3.8 Activity 4

Create a new .conf file called X-my-config.conf, where X is a number between 01-49. Set PasswordAuthentication no and restart sshd.

Can you connect as student? You cannot! Because sshd will read your config before 50-cloud-init.conf.

3.9 Activity 5

ssh and sshd are very configurable. Note that there is a line for Port that is commented out in /etc/ssh/sshd_config. Create a new file 02-port-config.conf and put the line Port 2222. Then restart your sshd and exit. Try to ssh in again - you cannot! You have changed the port to 2222.

```
root@droplet$ # use vim to edit /etc/ssh/sshd_config.d/02-port-config.conf
root@droplet$ grep "Port" /etc/ssh/sshd_config.d/02-port-config.conf
Port 2222
root@droplet$ systemctl restart sshd
root@droplet$ exit
me@myPC$ ssh root@droplet
# hangs, doesnt work
me@myPC$ ssh root@droplet -p 2222
# works!
```

Remove the config and restart sshd to continue. Unless you want to continue using port 2222, that's fine too!

3.10 Activity 6

SSH and sshd are very configurable. Look in **/etc/motd** to see the message of the day. This text is shown when you ssh in. Don't believe me? use vim or cat to see the contents of /etc/motd. Then disconnect and reconnect to your droplet. You will see that message.

Change this text, and reconnect to your droplet. You should see the message you created.

Now let's have some fun...

```
root@droplet$ apt install cowsay
root@droplet$ /usr/games/cowsay "welcome to my droplet!"
 ------------------------
< welcome to my droplet! >
 ------------------------
        \ ^__^
```

```
        \ (oo)\_______
           (__)\ )\/\
               ||----w |
               || ||
root@droplet$ /usr/games/cowsay "welcome to my droplet!" > /etc/motd
root@droplet$ exit
me@myPC$ ssh root@droplet
 -------------------------
< welcome to my droplet! >
 -------------------------
       \ ^__^
        \ (oo)\_______
           (__)\ )\/\
               ||----w |
               || ||
root@droplet$
```

4 SFTP

SFTP is the Secure File Transfer Protocol. It's a super useful tool for putting files on a server and getting them off. In this partr of the lab, we will get files off of the droplet, and put files onto the droplet.

4.1 How to use SFTP

```
me@myPC$ sftp root@droplet
# OR sftp -P 2222 root@droplet, if you didn't change the port back to 22.
# note the -P is capital
sftp> exit
me@myPC$
```

4.2 What SFTP can do

SFTP works alot like what you're used to in the shell, but

- It has a limited set of commands
- Some commands are local, they run on your pc. For example, lls.
- Some commands run on the remote, your droplet. For example, ls.

Local Command	Remote Command	What It Does
lls	ls	list files and directories
lcd	cd	change directory
lpwd	pwd	tell you your present working dir.
lmkdir	mkdir	make a directory

The other 2 commands you need to know are

1. get - download a file
2. put - upload a file

4.3 Activity 1

Let's upload a file to the droplet. Use vim in the terminal on your pc to create a file called poem.txt. Then upload it to /root/poetry.

```
me@myPC$ vim poem.txt
# create the file
me@myPC$ cat poem.txt
roses are red
violets are blue
me@myPC$ sftp root@droplet
sftp> lls
# use lls, lcd, lpwd to navigate to the location where the poem.txt file is saved.
# then use mkdir to create /root/poetry on the remote.
# then use cd to navigate to /root/poetry on the remote.
sftp> put poem.txt
# it uploads
sftp> exit
me@myPC$ ssh root@droplet
me@myPC$ cd /root/poetry
me@myPC$ ls
poem.txt
```

4.4 Activity 2

Try to upload a handful of other files onto your droplet. This is an absolutely critical skill. Make sure you are 100% comfortable with this.

4.5 Activity 3

Download a handful of files off of your droplet using the get command. Make sure you are 100% comfortable with this essential skill.

```
me@myPC$ sftp root@droplet
sftp> cd /etc/ssh/sshd_config.d
sftp> ls
50-cloud-init.conf
sftp> get 50-cloud-init.conf
sftp> exit
me@myPC$ ls
stuff 50-cloud-init.conf other-stuff
```

4.6 Activity 4

Let's have fun getting a tarball. Maybe you have to get 100 files off of a computer.. would be easier to download them as a single file rather than downloading 100 files.

```
me@myPC$ ssh root@droplet
root@droplet$ mkdir importantFiles
root@droplet$ echo hi > importantFiles/hi.txt
root@droplet$ echo bye > importantFiles/bye.txt
root@droplet$ ls importantFiles
bye.txt hi.txt
root@droplet$ tar cvf important.tar importantFiles
root@droplet$ ls
importantFiles important.tar
root@droplet$ tar --list -f important.tar
# this will just show you whats in the archive.
root@droplet$ exit
me@myPC$ sftp root@droplet
sftp> get important.tar
me@myPC$ ls
stuff important.tar other-stuff
```

Isn't that great! You just stuffed all the files in importantFiles into a tarball and downloaded them with the get command.

5 Final Activity

Now let's exercise what you learned. Can you configure 2 droplets A and B such that A can ssh into B as user "student" like:

Listing 1: Goal: Make this work

```
root@A$ ssh student@B
connected!
```

A rough sketch of how to do this:

1. ssh into A from your pc.
2. On droplet A, run ssh-keygen.
3. you need to get the public key from the A machine's .ssh directory into the .ssh/authorized_keys file on B
4. ssh into B from your pc. NOT FROM A!!!
5. On droplet B, add a user called student and set a password.
6. create a folder /home/student/.ssh on B with permissions 700
7. add a file called authorized_keys to /home/student/.ssh on B.
8. The authorized key file should have permission 600.

9. make sure that the .ssh folder and the authorized keys file on B are owned by student:student. You do this like `chown -R student:student /home/student/.ssh`

10. On droplet B, turn on password access for ssh so you're able to upload the pubkey.

11. sftp into B from A as student

12. Upload the pub key from A to B using the put command.

13. Note: You turned on password authentication, so you can get into B from A using the student password.

14. Then cat the pub key into /home/student/.ssh/authorized_keys on B.

15. now turn off password access on B.

16. if all went well you can now ssh into B from A without a password just as shown in Listing 1.